NAME:-Shubham Suryakant Biradar
ROLL NO.:-A-12
BATCH:-A1
ASSINGMENT:-2

Aim:-Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

Q.1.What is data ingestion and why is it important in data science? Which Python library is commonly used for data ingestion and cleaning? Answer:-
Data ingestion is the process of collecting, importing, and loading data from various sources (such as databases, CSV files, APIs, or cloud storage) into a system where it can be stored and analyzed. It is the first step in the data science pipeline.

Data ingestion is important because it ensures that data from different sources is gathered, organized, and made available for further processing. Without proper ingestion, data may be incomplete, inconsistent, or inaccessible, which can lead to incorrect analysis and poor decision-making. It also helps in maintaining data quality, consistency, and reliability, which are essential for accurate results.

A commonly used Python library for data ingestion and cleaning is Pandas. It provides functions like read_csv(), read_excel(), and read_sql() to easily import data, and powerful tools for cleaning and transforming data, such as handling missing values, filtering, and reshaping datasets.

Q.2.Why is data cleaning necessary before performing analysis or machine learning? **Answer:-**Data cleaning is necessary before performing analysis or machine learning because real-world data is often incomplete, inconsistent, and noisy. It may contain missing values, duplicate records, incorrect entries, or outliers, which can negatively affect the accuracy of results. If such issues are not handled, the analysis may produce misleading insights and machine learning models may learn incorrect patterns.

By cleaning the data, we ensure that it is accurate, consistent, and reliable, which improves the quality of analysis. It also helps in handling missing values, removing duplicates, correcting errors, and standardizing formats, making the dataset more suitable for processing. In machine learning, clean data leads to better model performance, higher accuracy, and more reliable predictions.

Q.3.How do you load a CSV file into a Pandas DataFrame? **Answer:-**You can load a CSV file into a Pandas DataFrame using the read_csv() function. **Example:-**import pandas as pd

Load CSV file

df = pd.read_csv('data.csv')

Display first 5 rows

print(df.head())

Q.4.What is the difference between head() and tail() functions in Pandas? **Answer:-**The head() and tail() functions in Pandas are used to quickly view a small portion of a dataset.

head(): This function displays the first few rows of the DataFrame. By default, it shows the first 5 rows, but you can specify any number (e.g., head(10) for first 10 rows). It is useful for quickly checking the beginning of the dataset. **tail():** This function displays the last few rows of the DataFrame. By default, it also shows the last 5 rows, and you can customize it (e.g., tail(10)). It is useful for viewing the end of the dataset.

Q.5.How can you identify missing values in a dataset? **Answer:-**1. Using isnull() or isna() These functions check each value in the dataset. They return True where data is missing and False where data is present. Example idea: df.isnull()→ shows missing values in the dataset 2. Using sum() with isnull() Counts total missing values in each column. Example idea: df.isnull().sum()→ gives number of missing values column-wise 3. Using info() function Provides a summary of the dataset. Shows number of non-null values, so you can detect missing data. Example idea: df.info() 4. Using describe() function Gives statistical summary. Missing values can be noticed by comparing count with total rows. Example idea: df.describe() 5. Visual inspection Sometimes you can directly look at the dataset. Missing values may appear as: NaN None empty cells

```
#CODE:-
#Handle missing values using mean or median. give me code
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'columna': [1, 2, 3, np.nan, 5],
    'columnb': [6, np.nan, 8, 9, 10],
    'columnc': [11, 12, 13, 14, np.nan],
    'columnd': ['A', 'B', 'C', 'D', 'E']
})

print("Original DataFrame:\n", df)
print("\nMissing values before imputation:\n", df.isnull().sum())
```

```
numeric_cols = df.select_dtypes(include=['number'])

df[numeric_cols.columns] = numeric_cols.fillna(numeric_cols.mean())

print("\nDataFrame after filling missing numeric values with mean:\n", df)
print("\nMissing values after imputation:\n", df.isnull().sum())
```

```
Original DataFrame:
    columna columnb columnc columnd
0      1.0       6.0     11.0      A
1      2.0       NaN     12.0      B
2      3.0       8.0     13.0      C
3      NaN       9.0     14.0      D
4      5.0      10.0      NaN      E

Missing values before imputation:
  columna   1
columnb    1
columnc    1
columnd    0
 dtype: int64

DataFrame after filling missing numeric values with mean:
    columna columnb columnc columnd
0     1.00      6.00    11.0       A
1     2.00      8.25    12.0       B
2     3.00      8.00    13.0       C
3     2.75      9.00    14.0       D
4     5.00     10.00    12.5       E

Missing values after imputation:
  columna   0
columnb    0
columnc    0
columnd    0
dtype: int64
```

```
#Change data types of selected columns.
df['columna'] = df['columna'].round().astype(int)
print("\nDataFrame after converting 'columna' to integer type:\n", df.dtypes

# Change 'columnd' to categorical type
df['columnd'] = df['columnd'].astype('category')
print("\nDataFrame after converting 'columnd' to categorical type:\n", df.dt

# Display the updated DataFrame
print("\nUpdated DataFrame:\n", df)
```

```
DataFrame after converting 'columna' to integer type:
  columna      int64
columnb     float64
columnc     float64
columnd    category
dtype: object

DataFrame after converting 'columnd' to categorical type:
 columna       int64
```

```
columnb     float64
columnc     float64
columnd    category
dtype: object

Updated DataFrame:
    columna columnb   columnc columnd
0         1    6.00     11.0       A
1         2    8.25     12.0       B
2         3    8.00     13.0       C
3         3    9.00     14.0       D
4         5   10.00     12.5       E
```

```
#Save the cleaned DataFrame to a new CSV file
df.to_csv('cleaned_data.csv', index=False)

print("Cleaned data saved to 'cleaned_data.csv'")
```

```
Cleaned data saved to 'cleaned_data.csv'
```

CONCLUSION:-Data ingestion and cleaning are essential steps in data analysis. Data ingestion helps in collecting data from different sources, while data cleaning ensures the data is accurate, consistent, and free from errors. By handling missing values, removing duplicates, and fixing inconsistencies, the dataset becomes reliable and ready for analysis.